# CSA1735 – Artificial Intelligence

# CO1 Assessment 1 (AT1)

**Student:** Kaushik Narayanan V | **Register No:** 192321047

**Slot A | Questions:** 4, 8, 12, 13, 14

### Q4. Route Finding with A* Search

**Evaluation function:** f(n) = g(n) + h(n), where g(n) is the path cost from the start and h(n) is the estimated cost to the goal. h must be admissible (never overestimates) and, for graph search, consistent.

**Assumed map (edge costs):** S–A = 1, S–B = 4, A–B = 2, A–C = 5, A–G = 12, B–C = 2, C–G = 3

**Heuristic (straight-line estimate):** h(S)=7, h(A)=6, h(B)=4, h(C)=2, h(G)=0

| Step | Expand | Generated (g, f) | Open list after step |
|---|---|---|---|
| 1 | S (f=7) | A (1,7), B (4,8) | A7, B8 |
| 2 | A (f=7) | B via A (3,7) improves 4→3; C | B7, C8, G13 |

| Step | Expand | Generated (g, f) | Open list after step |
|---|---|---|---|
| | | (6,8); G (13,13) | |
| 3 | B (f=7) | C via B (5,7) improves 6→5; | C7, G13 |
| 4 | C (f=7) | G via C (8,8) improves 13→8 | G8 |
| 5 | G (f=8) | Goal reached | – |

**Optimal path:** S → A → B → C → G, **cost = 8**. Nodes expanded: S, A, B, C, G.

**Properties:** Complete and optimal when h is admissible (tree search) or consistent (graph search). Time and space are exponential in the worst case, O(b^d). A* is optimally efficient: no other optimal algorithm using the same h expands fewer nodes.

### Q8. Uninformed Search Comparison (BFS, DFS, IDS)

Let b = branching factor, d = depth of the shallowest goal, m = maximum depth of the tree.

| Property | BFS | DFS | IDS |
|---|---|---|---|
| Strategy | Expands shallowest node first (FIFO queue) | Expands deepest node first (LIFO stack) | Repeated depth-limited DFS with limit 0, 1, 2, … |
| Complete? | Yes (finite b) | No (infinite or looping spaces); yes only in finite | Yes (finite b) |

| Property | BFS | DFS | IDS |
|---|---|---|---|
| | | acyclic spaces | |
| Optimal? | Yes, if step costs are equal | No | Yes, if step costs are equal |
| Time | O(b^d) | O(b^m) | O(b^d) |
| Space | O(b^d) | O(bm) | O(bd) |

**Key points**

- BFS finds the shallowest solution but needs memory that grows exponentially.
- DFS uses very little memory but can wander down deep or infinite branches and return a non-optimal path.
- IDS combines BFS completeness and optimality with DFS memory use. The repeated work is small because most nodes are at the deepest level: total nodes ≈ $(d+1)\cdot 1 + d\cdot b + \ldots + 1\cdot b^d = O(b^d)$.
- **Best choice:** IDS is the preferred uninformed method when the search space is large and the solution depth is unknown.

---

**Q12. Mars Rover Agent**

**Percepts**

Camera images, stereo/depth data, LIDAR/obstacle distance, wheel odometry, IMU (tilt, orientation), GPS/star-tracker position, temperature and radiation sensors, battery level, soil/rock spectrometer readings, radio commands from Earth.

**Environment**

Mars surface: rocky, sandy, uneven terrain, dust storms, extreme temperature, communication delay with Earth.

| Property | Type | Reason |
|---|---|---|
| Observability | Partially observable | Sensors have limited range; terrain beyond view is unknown |
| Agents | Single agent | Only the rover acts (Earth control is a supervisor) |
| Determinism | Stochastic | Wheel slip, dust and sensor noise give uncertain outcomes |
| Episodic/Sequential | Sequential | Current moves affect later safety and energy |
| Static/Dynamic | Dynamic | Weather, lighting and dust change while it deliberates |
| Discrete/Continuous | Continuous | Position, speed and time are continuous |
| Known/Unknown | Partly unknown | Terrain must be learned |

**Actions (actuators)**

Move forward/backward, turn, stop, climb or avoid obstacles, drill, scoop soil, move robotic arm, take photos, run experiments, deploy/orient solar panels, transmit data, enter sleep/safe mode.

**Performance measure**

Number and quality of scientific samples/data collected, area explored, mission success, safety (no tip-over, no collisions), energy efficiency, successful data transmission, time taken.

**Agent architecture**

A **utility-based, model-based agent with a layered architecture**:

1. **Sensors → state update:** the internal model of terrain, position and battery is updated from percepts.
2. **Reactive layer:** immediate reflexes (stop when tilt is too high, avoid an obstacle, enter safe mode on low battery).
3. **Planning layer:** path planning (A*), task scheduling for science targets.
4. **Utility function:** trades off science value, risk and energy.
5. **Actuators:** execute the chosen action; a **learning element** can improve terrain classification over time.

## Q13. 8-Queens Problem (Formulation and Search)

**Goal:** place 8 queens on an 8×8 board so that no two share a row, column or diagonal.

### Formulation (incremental)

- **States:** any arrangement of 0 to 8 queens on the board with none attacking another (placed column by column).
- **Initial state:** empty board.
- **Actions:** add a queen to the leftmost empty column in a square not attacked by any existing queen.
- **Transition model:** returns the board with the queen added.
- **Goal test:** 8 queens on the board, none attacked.
- **Path cost:** irrelevant; only the final state matters.

**State-space size:** the naive complete-state formulation has about 1.8 × 10^14 states (64·63·…·57). The incremental no-attack formulation reduces this to **2,057 states**.

### Search

**Backtracking (DFS with constraint checking):** place a queen in column 1, then column 2 in a safe row, and so on. If a column has no safe row, backtrack to the previous column and try the next row.

```
solve(col):
 if col == 8: return true
 for row in 0..7:
  if safe(row, col):
   place queen at (row, col)
```

```
    if solve(col+1): return true
    remove queen
 return false
```

safe(row, col) checks that no earlier queen shares the row or either diagonal.

**One solution** (row number of the queen in columns 1–8): **1, 5, 8, 6, 3, 7, 2, 4**

**Facts:** 92 distinct solutions, 12 unique up to rotation and reflection. Alternatives: BFS (memory heavy), hill climbing with min-conflicts (fast but can get stuck in local minima, about 14% success from random starts, fixed by random restarts or simulated annealing), and genetic algorithms.

## Q14. OLA Cab Booking – Problem-Solving Agent and Pseudocode

### Problem-solving agent

A goal-based agent that (1) formulates a goal, (2) formulates the problem, (3) searches for a sequence of actions, then (4) executes it while monitoring.

- **Goal:** the customer is picked up and dropped at the destination in the shortest time/cost.
- **Initial state:** customer at pickup location with a booking request; set of available cabs with their locations.
- **States:** (cab positions, customer status: waiting / picked / dropped, current location).
- **Actions:** assign a cab, accept/reject by driver, drive to the pickup, start trip, take a road segment, end trip, process payment.
- **Transition model:** result of an action, e.g. a cab moves along a road segment and its location updates.
- **Goal test:** the customer is at the destination and payment is complete.

- **Path cost:** travel time + distance + waiting time (fare/ETA); step cost depends on traffic.

**PEAS:** Performance = low wait time, low fare, safe and punctual ride. Environment = city roads, traffic, drivers, customers. Actuators = assignment, route display, notifications, payment module. Sensors = GPS, traffic data, customer requests, driver status.

**Pseudocode**

```
function OLA-BOOKING-AGENT(percept) returns an action
  persistent: seq (action sequence, initially empty),
              state (current world description),
              goal, problem

  state ← UPDATE-STATE(state, percept)   // GPS, traffic, cab availability
  if seq is empty then
     goal ← FORMULATE-GOAL(state)       // pickup → drop at destination
     problem ← FORMULATE-PROBLEM(state, goal)
     seq ← SEARCH(problem)              // e.g. A* on road graph + nearest-cab
matching
     if seq = failure then return NO-CAB-AVAILABLE
  action ← FIRST(seq)
  seq ← REST(seq)
  return action

function SEARCH(problem)
  cab ← argmin over available cabs of ETA(cab, pickup)
  route_to_pickup ← A*(cab.location, pickup)
```

```
  route_to_drop ← A*(pickup, destination)
  return [ASSIGN(cab)] + route_to_pickup + [START-TRIP] + route_to_drop +
[END-TRIP, PAYMENT]
```

If a driver rejects the booking or traffic changes sharply, the agent re-plans (the percept changes the state and the sequence is rebuilt).